

Lección 3: Integration Testing

Pruebas de Integración User-Centric

Agenda

- ¿Qué son Integration Tests?
 - Testing Library: User-Centric Philosophy
 - Patrones de Integration Testing
 - Context y Provider Testing
 - Testing Hooks y API Integration
-

Integration vs Unit vs E2E

Unit Tests	Integration Tests	E2E Tests
Component	User Flow	Full App
Fast (< 10ms)	Medium (~100–500ms)	Slow (1–10s)
Isolated	Connected	Real Environment
Mock Dependencies	Real Context	Real Backend
High Volume	Medium Volume	Low Volume

Integration = Sweet Spot

¿Qué es Testing Library?

Testing Library: Familia de bibliotecas para testear UI centradas en el usuario

```
// React Testing Library
import { render, screen } from '@testing-library/react'
```

```
import userEvent from '@testing-library/user-event'
```

Propósito:

- Buscar elementos como lo haría un usuario (texto, roles, labels)
- NO buscar por detalles de implementación (clases CSS, IDs internos)
- Forzar buenas prácticas de accesibilidad

Alternativas anteriores: Enzyme (implementación), Jest DOM (básico)

Queries: Buscar Elementos en el DOM

Query: Función para encontrar elementos en tests

```
render(<button>Add to Cart</button>)

// 3 tipos de queries:
screen.getByRole('button')      // Lanza error si no existe
screen.queryByRole('button')    // Retorna null si no existe
screen.findByRole('button')     // Espera (async) si no existe aún
```

Cuándo usar cada tipo:

- `getBy` : Elemento DEBE existir inmediatamente
 - `queryBy` : Verificar que elemento NO existe
 - `findBy` : Elemento aparecerá después (async)
-

Testing Library Philosophy

“The more your tests resemble the way your software is used, the more confidence they can give you.”

```
// ❌ Testing detalles de implementación (CSS selector)
expect(wrapper.find('.cart-item')).toHaveLength(2);
```

```
// ✅ Testing comportamiento del usuario (query para acceder + texto visible)
expect(screen.getAllByRole('listitem')).toHaveLength(2);
```

```
// Aún mejor: lo que el usuario VE
expect(screen.getByText("2 items in cart")).toBeInTheDocument();
```

Diferencia: No es QUÉ testeas, sino CÓMO lo buscas

Query Priority

Priority Order:

1. 🏆 **getByRole**: Más accesible (buttons, links, inputs)
2. 🏅 **getByLabelText**: Para formularios con labels
3. 🏅 **getByText**: Para contenido visible
4. 🏅 **getByTestId**: Último recurso

Simula cómo los usuarios encuentran elementos

Patrón: Component Integration

```
describe('ProductCard Integration', () => {
  it('should add item to cart on click', async () => {
    const user = userEvent.setup()

    render(
      <CartProvider>
        <ProductCard product={mockProduct} />
      </CartProvider>
    )

    await user.click(
      screen.getByRole('button', { name: /add to cart/i })
    )

    expect(screen.getByText('Added to cart')).toBeVisible()
  })
})
```

Patrón: Form Integration (1/2)

```
describe('Checkout Form', () => {
  it('should complete checkout with valid data', async () => {
    const user = userEvent.setup()
    const mockSubmit = vi.fn()

    render(<CheckoutForm onSubmit={mockSubmit} />)

    // Fill form
    await user.type(screen.getByLabelText(/email/i), 'user@test.com')
    await user.type(screen.getByLabelText(/name/i), 'John Doe')
```

Patrón: Form Integration (2/2)

```
    // Submit
    await user.click(screen.getByRole('button', { name: /place order/i }))

    expect(mockSubmit).toHaveBeenCalledWith({
      email: 'user@test.com',
      name: 'John Doe'
    })
  })
})
```

API Integration con Vitest Mocks (1/2)

Patrón: Mock global + Once para casos específicos (success/error)

```
// Mock del service layer (setup global)
import { vi } from "vitest";
import { cartService } from "../services/cartService";

vi.mock("../services/cartService", () => ({
  cartService: {
    getProducts: vi.fn(),
    addToCart: vi.fn(),
  },
```

```
});

// Mock por defecto (happy path)
beforeEach(() => {
  vi.mocked(cartService.getProducts).mockResolvedValue({
    success: true,
    data: [{ id: "laptop", name: "Gaming Laptop", price: 899.99 }],
  });
});
```

API Integration con Vitest Mocks (2/3)

```
// Override en tests específicos
describe('Product Loading', () => {
  it('should handle network error', async () => {
    vi.mocked(cartService.getProducts).mockRejectedValueOnce(
      new Error('Network error')
    )
    render(<ProductList />)
    expect(await screen.findByText('Failed to load products')).toBeInTheDoc
  })
})
```

API Integration con Vitest Mocks (3/3)

```
it('should load empty product list', async () => {
  vi.mocked(cartService.getProducts).mockResolvedValueOnce({
    success: true, data: []
  })
  render(<ProductList />)
  expect(await screen.findByText('No products available')).toBeInTheDocum
})
it('should load products with default mock', async () => {
  render(<ProductList />)
  expect(await screen.findByText('Gaming Laptop')).toBeInTheDocument()
})
})
```

Testing Async Operations (1/2)

```
describe('Async Operations', () => {
  it('should show loading state during API call', async () => {
    render(<ProductSearch />)

    await user.type(
      screen.getByPlaceholderText('Search...'),
      'laptop'
    )

    // Loading appears
    expect(screen.getByText('Searching...')).toBeInTheDocument()
```

Testing Async Operations (2/2)

```
    // Wait for results
    await waitFor(() => {
      expect(screen.getByText('Gaming Laptop')).toBeInTheDocument()
    })

    // Loading disappears
    expect(screen.queryByText('Searching...')).not.toBeInTheDocument()
  })
})
```

Métricas de Integration Testing

Coverage Goals:

- User Flows: 90% de flujos principales
 - Component Interaction: 85%
 - State Management: 95%
 - API Integration: 80%
 - Form Validation: 100%
-

Ejercicio 1: Query por Role

Prompt:

Actúa como un desarrollador que practica testing user-centric.

CONTEXT0: Testing Library prioriza queries que simulan cómo usuarios reales interactúan con la app. Query Priority: 1) getByRole (más accesible), 2) getByLabelText, 3) getByText, 4) getByTestId (último recurso). getByRole busca elementos por su rol ARIA, como lo haría un screen reader (lector de pantalla para usuarios con discapacidad visual).

TAREA: Crea componente LikeButton y su integration `test`.

COMPONENTE REQUIREMENTS:

- Estado inicial: likes = 0
- UI: botón con texto "Like (0)"
- Acción: click incrementa likes y actualiza texto a "Like (1)"
- Framework: React con useState

TEST REQUIREMENTS:

- Framework: Vitest + Testing Library
- Imports: render, screen, userEvent desde @testing-library/react
- Query: getByRole('button', { name: /like \(\0\)/i })
- NO uses getByTestId (menos semántico)
- Interacción: userEvent.click() para simular click real
- Assertion: verificar texto cambió a "Like (1)"
- Estructura: AAA pattern

ARCHIVOS:

- src/features/likes/LikeButton.tsx (componente)
- src/features/likes/LikeButton.test.tsx (`test`)

VALIDACIÓN: ejecuta `pnpm test` para verificar

Aprende: getByRole es más semántico y accesible que getByTestId

Ejercicio 2: User Event Simulation

Prompt:

Actúa como un desarrollador que testa flujos de formularios con userEvent.

CONTEXT0: `userEvent` simula interacciones reales del usuario mejor que `fireEvent`. Por ejemplo, `userEvent.type()` dispara eventos `keyDown`, `keyPress`, `keyUp`, `input`, `change` (como usuario real), mientras `fireEvent.change()` solo dispara `change`. Query Priority #2 es `getByLabelText`, ideal para formularios con `<label>` asociados a `inputs` (mejor accesibilidad).

TAREA: Crea componente `PriceCalculator` con formulario y su integration `test`

COMPONENTE REQUIREMENTS:

- 2 inputs tipo number: `"Quantity"` y `"Unit Price"`
- Cada input debe tener `<label>` asociado (para `getByLabelText`)
- Cálculo: `total = quantity × unitPrice`
- UI: párrafo mostrando `"Total: $XX.XX"` (formato 2 decimales)
- Framework: React con `useState`

TEST REQUIREMENTS:

- Framework: Vitest + Testing Library
- Setup: `const user = userEvent.setup()`
- Query: `getByLabelText(/quantity/i)` y `getByLabelText(/unit price/i)`
- Interacción: `await user.type()` para ingresar valores
- Test data: `quantity = 3`, `unitPrice = 10.50`
- Assertion: verificar `"Total: $31.50"` en el documento
- Estructura: AAA pattern

ARCHIVOS:

- `src/features/calculator/PriceCalculator.tsx` (componente)
- `src/features/calculator/PriceCalculator.test.tsx` (`test`)

VALIDACIÓN: ejecuta `pnpm test` para verificar

Aprende: `userEvent` simula interacciones reales del usuario

Puntos Clave

1. **Sweet Spot** (punto óptimo): Balance perfecto entre unit y E2E
2. **Centrado en el Usuario:** Testa como usuario ve la app
3. **Interacciones Reales:** Componentes trabajando juntos
4. **Retroalimentación Rápida:** Más rápido que E2E
5. **Valor de Negocio:** Valida flujos reales de usuario
6. **Testing Library:** Herramienta clave para testing user-centric